ECOLE NATIONALE DES SCIENCES APPLIQUEES

Beni Mellal – Filiere IACS

Pr. BE ELBAGHAZAOUI

COMPTE RENDU

TP N°1

Application Monolithique avec Spring Boot

Gestion de Produits – Architecture en Couches

Etudiant :	Achraf HAMOUDA
Module :	Architecture Logicielle
Date :	30 Mars 2026
Encadrant :	Pr. BE ELBAGHAZAOUI

Annee Universitaire 2025–2026

Contents

1 Objectifs **2**

2 Architecture en Couches **2**

3 Implementation **2**
3.1 Configuration 2
3.2 Entite Product 2
3.3 Repository 3
3.4 Service 3
3.5 Controller REST 4

4 Endpoints REST **4**

5 Tests avec Postman **4**
5.1 Test 1 – Creer un produit (POST /api/products) 5
5.2 Test 2 – Lister les produits (GET /api/products) 5
5.3 Recapitulatif 6

6 Exercice – Entite Category **6**

7 Avantages et Limites du Monolithe **7**

8 Conclusion **7**

1. Objectifs

Ce TP vise a creer une **application Spring Boot monolithique** avec une API REST CRUD complete pour la gestion de produits, en appliquant l'architecture en couches classique.

- Creer une application Spring Boot monolithique simple
- Developper une API REST CRUD complete
- Comprendre l'architecture en couches (Entity, Repository, Service, Controller)
- Decouvrir `JpaRepository` pour simplifier l'acces aux donnees
- Identifier les avantages et limites d'un monolithe

2. Architecture en Couches

Couche	Classe	Role
Entity	`Product`	Modele JPA : id, name, description, price, stock
Repository	`ProductRepository`	JpaRepository – acces base de donnees PostgreSQL
Service	`ProductService`	Logique metier : CRUD, validation, transactions
Controller	`ProductController`	Endpoints REST sur `/api/products`

3. Implementation

3.1 Configuration

Listing 1. application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/ecommerce
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

3.2 Entite Product

Listing 2. Product.java – Entite JPA

```
@Entity
@Table(name = "products")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

```

```java
    @NotBlank(message = "Product name is required")
    @Column(nullable = false)
    private String name;

    @Column(length = 1000)
    private String description;

    @NotNull @Positive
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @NotNull @PositiveOrZero
    @Column(nullable = false)
    private Integer stock;
}
```

3.3 Repository

Listing 3. ProductRepository.java

```java
@Repository
public interface ProductRepository extends JpaRepository<Product,
    Long> {
    List<Product> findByNameContainingIgnoreCase(String name);
}
```

3.4 Service

Listing 4. ProductService.java – Logique metier

```java
@Service
@RequiredArgsConstructor
@Transactional
public class ProductService {
    private final ProductRepository repository;

    public List<Product> getAll() { return repository.findAll(); }

    public Product getById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException(
                "Product not found: " + id));
    }

    public Product create(Product product) { return repository.save(
        product); }

    public Product update(Long id, Product details) {
        Product product = getById(id);
        product.setName(details.getName());
        product.setPrice(details.getPrice());
        product.setStock(details.getStock());
        return product;
    }

    public void delete(Long id) {
```

```
        repository.delete(getById(id));
    }
}
```

3.5 Controller REST

Listing 5. ProductController.java – Endpoints REST

```
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService service;

    @GetMapping
    public ResponseEntity<List<Product>> getAll() {
        return ResponseEntity.ok(service.getAll());
    }

    @PostMapping
    public ResponseEntity<Product> create(@Valid @RequestBody Product
        p) {
        return ResponseEntity.status(HttpStatus.CREATED).body(service
            .create(p));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Product> update(@PathVariable Long id,
                                          @Valid @RequestBody Product
                                              p) {
        return ResponseEntity.ok(service.update(id, p));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

4. Endpoints REST

Methode	URL	Description
GET	/api/products	Liste tous les produits
POST	/api/products	Creer un produit
PUT	/api/products/{id}	Modifier un produit
DELETE	/api/products/{id}	Supprimer un produit

5. Tests avec Postman

5.1 Test 1 – Creer un produit (POST /api/products)

```
POST http://localhost:8080/api/products
Content-Type: application/json
{ "name": "Laptop Dell", "price": 9999.99, "stock": 5 }
```

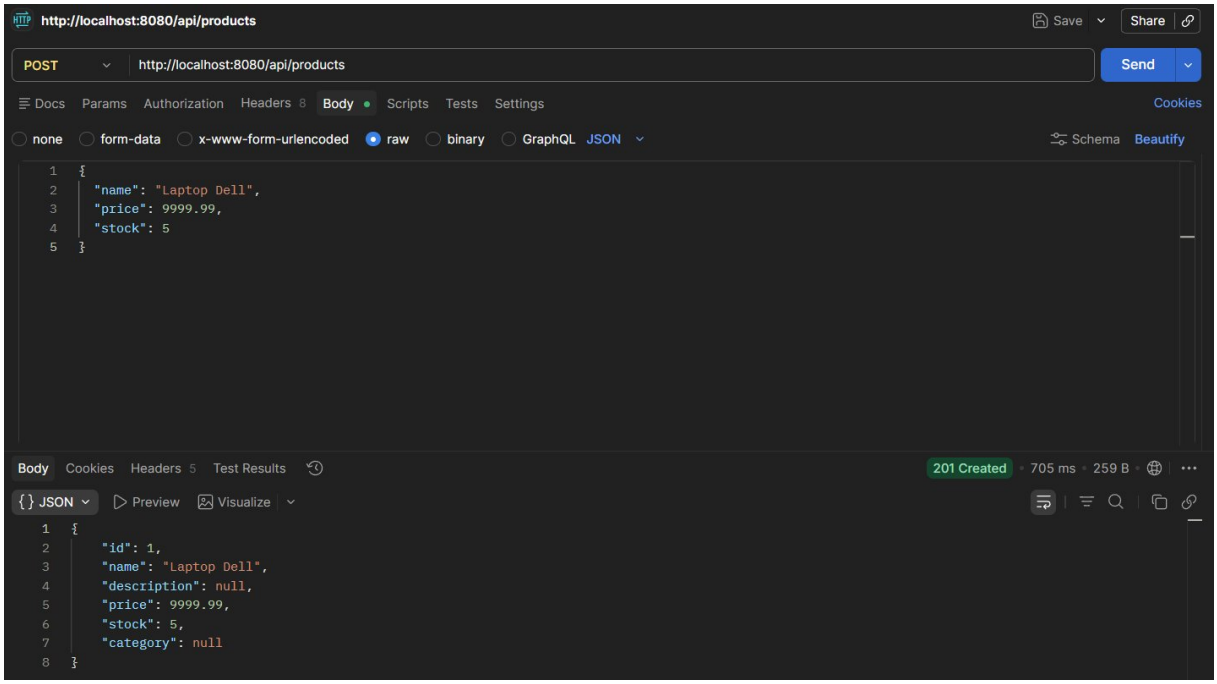

Figure 1. POST /api/products – 201 Created, produit cree avec id=1

Resultat

`HTTP 201 Created` – Produit cree avec id=1, price=9999.99, stock=5. Persiste en base PostgreSQL.

5.2 Test 2 – Lister les produits (GET /api/products)

```
GET http://localhost:8080/api/products
```

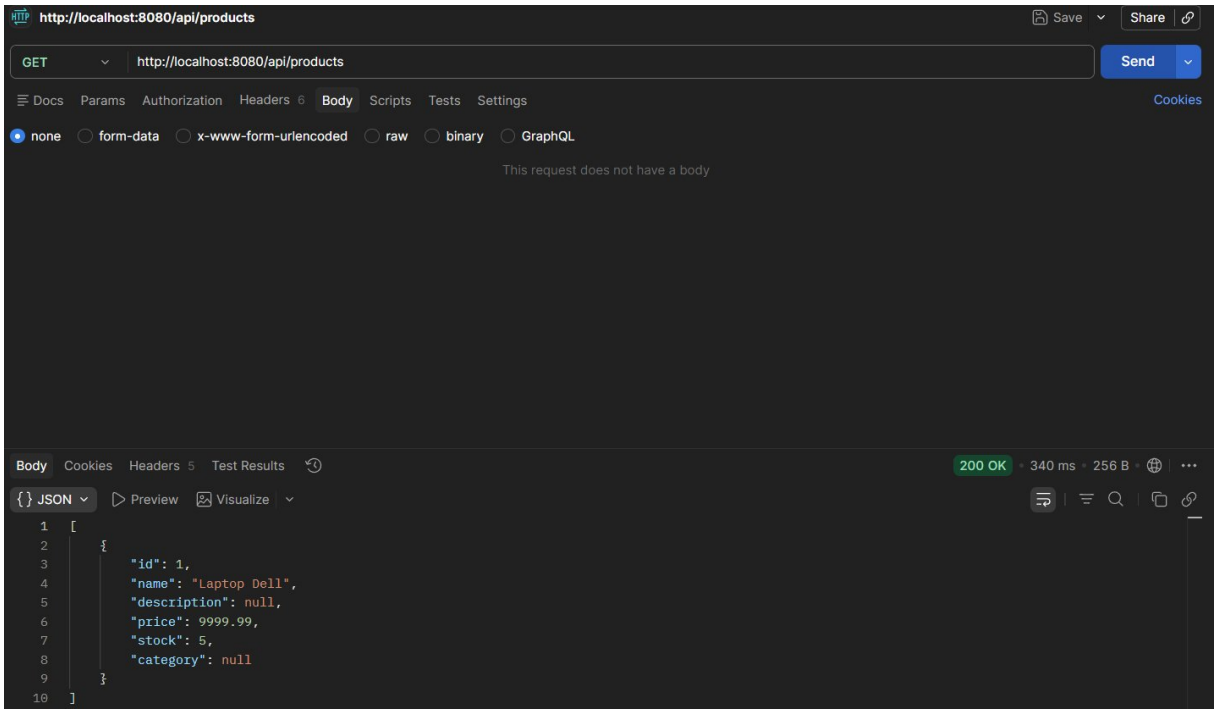

Figure 2. GET /api/products – 200 OK, liste des produits retournee

Resultat

`HTTP 200 OK` – Liste retournee avec Laptop Dell (id=1, price=9999.99, stock=5).

5.3 Recapitulatif

Test	Methode	URL	Code
Creer produit	POST	`/api/products`	201
Lister produits	GET	`/api/products`	200

6. Exercice – Entite Category

L'exercice demande d'ajouter une entite **Category** avec une relation `@ManyToOne` avec `Product`.

Listing 6. Category.java + relation dans Product

```
@Entity
public class Category {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
}

// Dans Product.java, ajouter :
@ManyToOne
@JoinColumn(name = "category_id")
private Category category;
```

7. Avantages et Limites du Monolithe

Avantages	Limites
Simple a deployer (un seul JAR)	Difficile a scaler independamment
Facile a deboguer et tester	Couplage fort entre modules
Transactions ACID simples	Un bug peut faire tomber tout le systeme
Performances elevees (pas de reseau)	Migration difficile vers microservices

8. Conclusion

Ce TP a permis de mettre en place une application Spring Boot monolithique complete avec l'architecture en couches classique. L'approche Entity/Repository/Service/-Controller garantit une bonne separation des responsabilites et prepare le terrain pour une future refactorisation en modules metier (TP2) puis en microservices (TP4).